

AI ENGINEER TAKE-HOME CASE STUDY

OVERVIEW

Build a RAG system that answers questions over a corpus of semiconductor equipment company earnings materials. This is a simplified version of one of the internal research tools you would build in the role.

The system must answer questions using only the provided documents, cite its sources, and explicitly decline to answer when the documents do not contain the answer. A hallucinated answer is significantly worse than a refusal.

PROVIDED MATERIAL

Earnings releases and earnings call transcripts (PDFs) for Lam Research, KLA, Applied Materials, and ASML, covering periods in 2024–25.

Note: these companies have different fiscal calendars. Handle dates and periods carefully.

CORE REQUIREMENTS

1. Grounded answers only. Every answer must be derived from the provided corpus. Do not answer from the model's general knowledge, even when it "knows" the answer.
2. Source attribution on every answer. Each answer must include: source document, location (page or section), and a short supporting quote.
3. Explicit abstention. If the corpus does not contain the answer, the system must say so clearly (eg "Not found in the provided documents") rather than guess. Abstention must hold even for questions a general LLM could answer.
4. Numeric robustness. Some answers exist only in the financial tables of the earnings releases, not in the earnings transcript. Your system should handle these.
5. Re-runnable ingestion. Ingestion must work on an arbitrary folder of PDFs without code changes. New documents will be added to the corpus after submission.

REQUIRED DELIVERABLES

Submit a **zip file** containing:

project/

README.txt # setup + run instructions
requirements.txt # pinned dependencies
src/ # your code
.env.example # required API keys, values blank (do not include real keys)
writeup.pdf # one page, see below
data/index/ # pre-built index

Your system must run exactly these commands:

python ingest.py --input <folder> # (re)build the index from a folder of PDFs
python ask.py "<question>" # answer a single question

Input:

To evaluate your case study, we will run ask.py against a list of questions, one question at a time. Each run of ask.py must complete without manual intervention and print its result to the console.

Your system will face questions like these (the evaluation set is different but similar in style):

1. What net bookings did ASML report for Q3 2024, and how did management characterise them on the call?
2. Which companies quantified China as a share of revenue in their December 2024 quarterly results, and what were the figures?
3. What was KLA's services revenue in its December 2024 quarter, per the earnings release?
4. Did Lam Research's quarterly revenue guidance increase or decrease between its Fiscal 2Q25 and Fiscal 3Q25 results, and by how much?
5. What capex guidance did TSMC give for 2025? (Expected behaviour: abstain as TSMC's earnings materials are not included in the provided documents).

Required output format:

ANSWER: <the answer>

CITATIONS:

- <document name>, p.<page>: "<short supporting quote>"

or, when the corpus does not contain the answer:

ABSTAINED: Not found in the provided documents.

Every response must contain either an answer with at least one citation, or an explicit abstention. We evaluate correctness and citation validity on answerable questions. Uncited answers or guesses dressed as an answer will be marked poorly. We will also evaluate retrieval quality, including handling of financial tables.

Optional (secondary)

- A lightweight UI (eg Streamlit/Gradio) is welcome but not required. We evaluate for correctness in the answers of the system, so do not spend disproportionate time on the front end.

The one-page writeup should include a summary of the following:

- Architecture and key design choices
- Model used and why, including any cost & latency considerations
- Limitations of this system
- What you would build next with more time
- Where you used AI coding assistants

GUIDANCE

- AI coding assistants (Claude Code, Cursor, Codex, etc) are permitted and encouraged. Flag in your writeup where you used them.
- You can use any of the main model providers. We do not supply API credits. If you decide to use a chargeable API, spend should be well under £10. Google's Gemini API offers an ongoing free tier sufficient for this task. State your model choices and rationale in the writeup.
- Assume the production version of this application indexes thousands of documents. Your architecture should be the one you'd choose at that scale.
- Evaluation questions relate to the provided earnings materials only. Some questions are deliberately unanswerable from the documents provided.
- We will run your system on a Windows machine, following only your README. Use OS agnostic architecture and include instructions compatible with Windows.

NEXT STEPS

Following review, you will attend a live session. During the session you will run your system on your own laptop, walk us through it, and extend it live with additional documents we provide on the day. Bring the system in runnable state with working API access.

LOGISTICS

- Submission: via email to Richard Walker (rwalker@crakeam.com) and Yannis Chatzilakos (ychatzilakos@crakeam.com)
- Deadline: Sunday 19/07/26 23:00
- Questions during the exercise: via email to Richard Walker (rwalker@crakeam.com) and Yannis Chatzilakos (ychatzilakos@crakeam.com). Clarifying questions are welcome and not penalised.